
Regular Expressions

A Regular Expression (Regex) is a pattern used to search and match specific patterns in a string(location).

In python, regex operations are performed using the **re module**.

Before using regex in Python, import the **re module**.

Syntax: import re

Important Regex Functions in Python

1. re.search()

> Searches the first occurrence of the pattern anywhere in the location.

> If the pattern is found, it returns a match object with span(), otherwise it returns None.

> **span()**: it is a method which returns the start and end index of a given pattern if it is present in a location.

Syntax:

import re

re.search(par1 , par2)

par1 : pattern to be searched

par2 : where to search(location or string)

example:

```
import re
```

```
text = "Python is easy,Python is powerfull"
```

```
res1= re.search("Python", text)
```

```
print(res1) # <re.Match object; span=(0, 6), match='Python'>
```

```
res2= re.search("Java", text)
```

```
print(res2) #None
```

2. re.findall()

>The re.findall() function is used to find all occurrences of a pattern in a string(location).

>It returns a list of matches. If no match is found, it returns an empty list [].

Syntax: import re

re.findall(par1 , par2)

par1 : pattern to be searched

par2 : where to search(location or string)

example:

```
import re
```

```
text = "Python is easy, Python is powerful"
```

```
res1 = re.findall("Python", text)
```

```
print(res1) # ['Python', 'Python']
```

```
res2 = re.findall("Java", text)
```

```
print(res2) # [ ]
```

3. re.sub()

The sub() function is used to replace occurrences of a pattern in a string with another string.

Syntax: import re

re.sub(par1, par2, par3, par4)

par1 - pattern to search

par2 - String to replace

par3 – where to search(location or string)

par4 - Optional; maximum number of replacements (default 0 = all occurrences)

Example 1:

```
import re
text = "Python is easy. Python is powerful."
result = re.sub("Python", "Java", text)
print(result) # Java is easy. Java is powerful.
```

Example 2:

```
import re
text = "Python Python Python"
result = re.sub("Python", "Java", text, count=2)
print(result) # Java Java Python
```

4. re.fullmatch()

fullmatch() function is used to check whether the entire string matches the given pattern.

Syntax: re.fullmatch(pattern, string)

Returns a Match object if the whole string matches the pattern.
Returns None if even a single character does not match.

Example 1

```
import re
result = re.fullmatch("abc", "abc")
print(result)
```

Output: <re.Match object; span=(0, 3), match='abc'>
The entire string "abc" matches the pattern "abc".

Example 2

```
import re
result = re.fullmatch("abc", "abcd")
print(result)
```

Output: None
The string has an extra character (d), so the whole string does not match.

Meta characters:

These are special symbols in regular expressions that have special meanings.

1. \ (backslash)

- used to create a special sequence
- used to remove the special meaning of meta char

2. (dot)

>It is called as wildcard char.

>It matches any single character except a newline (\n).

>It is commonly used when you want to match any character at a specific position in a pattern.

Example 1:

```
text = "Hello World"
res = re.search(r"H.llo", text)
if res != None:
    print("Match found")
else:
    print("No match")
```

#Match found

Example 2:

```
import re
text = "I love Python"
res = re.search(r"I.lovz", text)
if res != None:
    print("Match found")
else:
    print("No match")
```

#No match

3. ^ (Caret)

The meaning of ^ depends on where it is used in the regular expression.

1. ^ Outside Square Brackets []

When ^ is used outside square brackets, it represents the beginning (start) of the string.

Syntax: ^pattern

Example 1

```
import re
s = "Python"
result = re.search(r"^Py", s)
print(result)
```

Output : <re.Match object>

Example 2

```
import re
s = "I Love Python"
result = re.search(r"^Py", s)
print(result)
```

Output: None

2. ^ Inside Square Brackets []

When ^ is the first character inside square brackets, it means NOT or EXCEPT. It matches every character except the characters mentioned inside the brackets.

Syntax: [^characters]

Example 1

```
import re
s = "py'th\"o(e>.m"
result = re.sub(r"^[A-Za-z0-9]", "", s)
print(result)
```

Output: pythoem

Example 2

```
import re
s = "Python123@# $"
print(re.sub(r"^[0-9]", "", s))
```

Output: 123

Example 3

```
import re
s = "Python123@# $"
print(re.sub(r"^[A-Za-z]", "", s))
```

Output: Python

4.\$(dollar)

>Matches the end of a string.

Syntax: pattern

Example: `re.findall(r"end$", "this is the end")` # ['end']

5. * (Star)

> Matches 0 or more repetitions of the preceding character.

Syntax: character***Example:**

```
print(re.findall(r"ab*", "a ab abb abbb")) #Output: ['a', 'ab', 'abb', 'abbb']
```

Explanation: b can appear 0 or more times.

6. + (Plus)

>Matches 1 or more repetitions.

Syntax: character+

Example:

```
print(re.findall(r"ab+", "a ab abb abbb")) #Output: ['ab', 'abb', 'abbb']
```

Explanation: At least one b must exist.

7. ? (Question Mark)

>Matches 0 or 1 occurrence (optional).

Syntax: character?

Example

```
print(re.findall(r"colou?r", "color colour")) #Output: ['color', 'colour']
```

Explanation: u is optional.

8. {} (Curly Braces)

>Specifies exact number of repetitions.

>Syntax: char{n} / char{min,max}

Example 1:

```
print(re.findall(r"a{2}", "a aa aaa aaaa")) #Output: ['aa', 'aa', 'aa']
```

Example 2:

```
print(re.findall(r"a{2,3}", "a aa aaa aaaa")) #Output: ['aa', 'aaa', 'aaa']
```

9. [] (Square Brackets)

>Matches any one character inside the brackets.

Syntax: [characters]

Example:

```
print(re.findall(r"[abc]", "apple banana cat")) #['a', 'a', 'a', 'a', 'b', 'a', 'a', 'c', 'a']
```

Meaning: Match a OR b OR c.

Range example

[a-z]: Matches any lowercase letter.

[A-Z]: Matches any uppercase letter.

[0-9]: Matches any digit.

Example:

```
import re
text = "Hello WORLD Python"
res = re.findall(r"[A-Z]", text)
print(res) #Output: ['H', 'W', 'O', 'R', 'L', 'D', 'P']
```

Explanation: [A-Z] finds every uppercase letter.

10. | (Pipe)

>Acts as OR operator.

Syntax: pattern1 | pattern2**Example:**

```
print(re.findall(r"cat|dog", "cat dog tiger")) #Output: ['cat', 'dog']
```

Explanation: Match cat or dog.

11. () (Parentheses)

>Used for grouping patterns.

Syntax: (pattern)**Example:**

```
re.findall(r"(ab)+", "ab abab ababab") #Output: ['ab', 'ab', 'ab']
```

Explanation: Group ab and repeat it.

Anchor sequences:

\A = checks whether pattern is present in beginning or not

\Z = checks whether pattern is present in ending or not

\b = checks whether pattern present in begin or ending

\B = checks whether pattern not present in begin or ending

Q. Difference between ^ and \A in the re module of Python:

^ :

- It checks whether the pattern is present at the beginning of a line.
- It can match the start of each line in multiline strings.

\A:

- It checks whether the pattern is present at the beginning of the entire string.
 - It matches only once, at the very beginning of the string.
-

Q. Difference between \$ and \Z in the re module of Python:

\$:

- It checks whether the pattern is present at the end of a line.
- It can match the end of each line in multiline strings.

\Z:

- It checks whether the pattern is present at the end of the entire string.
 - It matches only once, at the very end of the string.
-

Creating special sequences:

```
import re
```

```
text = "This## is A## string$$ 12345 %%. special$$ chars## 6789 in this 456 90"
```

1. Digits (\d) – Returns all numbers

```
re.findall(r"\d+", text)
```

```
Output: ['12345', '6789', '456', '90']
```

2. Non-Digits (\D) – Returns all characters except numbers

```
re.findall(r"\D+", text)
```

```
Output: ['This## is A## string$$ ', ' %%. special$$ chars## ', ' in this ', ' ']
```

3. Word Characters (\w) – Letters and numbers

```
re.findall(r"\w+", text)
```

```
Output: ['This', 'is', 'A', 'string', '12345', 'special', 'chars', '6789', 'in', 'this', '456', '90']
```

4. Non-Word Characters (\W) – Special characters

```
re.findall(r"\W+", text)
```

```
Output: ['## ', ' ', '## ', '$$ ', ' %%. ', '$$ ', '## ', ' ', ' ', ' ', ' ']
```

5. Whitespace (\s) – Spaces

```
re.findall(r"\s+", text)
```

```
Output: [' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

6. Non-Whitespace (\S) – All characters except spaces

```
re.findall(r"\S+", text)
```

```
Output: ['This##', 'is', 'A##', 'string$$', '12345', '%%. ', 'special$$', 'chars##', '6789', 'in', 'this', '456', '90']
```

1. Validating email:

Structure of an Email Address Format: username@domain.extension

Example: poornima123@gmail.com

- Username → poornima123
- Domain name → gmail
- Extension → com

1. Username Rules (before @)

Username CAN have:

- Letters → a-z, A-Z
- Numbers → 0-9
- Underscore → _
- Dot → .
- Hyphen → -
- Consecutive _ allowed? Yes.

Username CAN start with:

- Letter
- Number
- _

Username SHOULD NOT:

- Start with . and -
- Have consecutive dots .. and - -
- End with . and -
- Have spaces

2. Domain Name Rules (between @ and extension)

Domain name CAN have:

- Letters
- Numbers
- Hyphen -
- Dot . (for subdomains)

Domain name SHOULD NOT:

- Have _
- Start with - and .
- End with – and .
- Have spaces
- Have consecutive dots ..

Consecutive - - allowed? Yes

3. Extension Rules (after last .)

Extension CAN have:

- Only letters usually

Extension SHOULD NOT:

- Contain numbers
 - Contain special characters
 - Be empty
-

Ex1:

```
import re
mail = "Pyspiders_2026@Py-btm.edu"

res = re.search( r "[A-Za-z0-9_]+@[A-Za-z-]+\.(com|in|edu|org)" ,mail)

if res != None:
    print("Valid email")
else:
    print("Invalid mail")
```

Explanation:**1.[A-Za-z0-9_]+**

- Matches one or more characters that can be uppercase letters (A–Z), lowercase letters (a–z), digits (0–9), or underscore (_).
- Represents the username part of the email.

2. @

- Matches the @ symbol which separates the username and domain name in an email.

3. [A-Za-z-]+

- Matches one or more uppercase or lowercase letters and hyphen.
- Represents the domain name.

4. \.

- Matches a dot (.) character. Used before the domain extension.

5. (com|in|edu|org)

- Matches any one of the given extensions: com, in, edu , or org.
-

Ex2: With re.fullmatch()**Ex:**

```
import re
```

```
e1 = "Anamika.1996@gmail.com"
```

```
r_pat = r'^[A-Za-z0-9_]+[A-Za-z0-9_]*@[A-Za-z0-9-]+\.(com|edu|org|in)$'
```

the above pattern will work with consecutive dots also ,hence we go with next pattern

```
r_pat1 = r'^[A-Za-z0-9_]+(\.[A-Za-z0-9_]*)@[A-Za-z0-9-]+\.(com|edu|org|in)$'
```

```
res = re.fullmatch(r_pat1,e1)
```

```
print(res)
```

```
if res != None:
```

```
    print("Valid Email")
```

```
else:
```

```
    print("Invalid Email")
```

Explanation:**1. ^**

Start of the string. Means email must start from here (no extra characters before)

2. [A-Za-z0-9_]+

Username must start with:

- letters (A–Z, a–z) , numbers (0–9)

At least one character required

3. (\.[A-Za-z0-9]+)*

This is the important part (handles dots properly)

- \. → a dot
- [A-Za-z0-9]+ → followed by letters/numbers
- () → grouping
- * → can repeat 0 or more times

But dot must always be followed by characters

Valid:

- anamika
- anamika.1996
- a.b.c

Invalid:

- .anamika (cannot start with dot)
- anamika. (cannot end with dot)
- ana..mika (no consecutive dots)

4. @: Mandatory symbol separating username and domain

5. [A-Za-z0-9]+ :Domain name:

- letters
- numbers
- hyphen -

6. \. : Dot before extension(escaped because . is special outside [])

7. (com|edu|org|in): any one domain extension

8. \$: End of string. Means nothing extra is allowed after extension

2. Validating Phone number:

- Here we are not validating length of phone number and length of country code.
- And also not validating whether country code exist or not.

Example:

```
import re
```

```
n1 = "+1 987 654 3210"
```

```
n2 = "+123-987-654-3210"
```

```
n3 = "123-987-654-3210"
```

```
res = re.fullmatch(r'\+?\d{1,3}[\s-]?\d{3}[\s-]?\d{3}[\s-]?\d{4}', n3)
```

```
if res != None:
```

```
    print("Valid number")
```

```
else:
```

```
    print("Invalid number")
```

Explanation:

- `\+?` → Optional + sign (for country code)
 - `\d{1,3}` → 1 to 3 digits (country/area code)
 - `[\s-]?` → Optional **space or hyphen**
 - `\d{3}` → 3 digits
 - `[\s-]?` → Optional **space or hyphen**
 - `\d{3}` → Next 3 digits
 - `[\s-]?` → Optional **space or hyphen**
 - `\d{4}` → Last 4 digits
-

3.Validating date:

date formats:

"dd-mm-yyyy" or "dd/mm/yyyy" or "yyyy-mm-dd" or "yyyy/mm-dd" or "dd.mm.yyyy"

Example 1:

```
import re
date1 = "05-03-2026"
res = re.fullmatch(r"\d{2}[-/]\d{2}[-/]\d{4}", date1)
if res != None:
    print("Valid Date")
else:
    print("Invalid Date")
Output: Valid Date
```

Explanation:

- **\d{2}**: Matches exactly 2 digits (Date).
Example: 05
 - **[-/]**: Matches either - or / as the separator.
 - **\d{2}**: Matches 2 digits for the month.
Example: 03
 - **[-/]**: Again matches - or /.
 - **\d{4}**: Matches 4 digits for the year.
-

Example 2:

```
import re
date2 = "45-13-2039"
res = re.fullmatch(r"\d{2}[-/]\d{2}[-/]\d{4}", date2)
if res != None:
    print("Valid Date")
else:
    print("Invalid Date")
Output: Valid Date : But it is not valid there is no 45 date and 13 month.
```

Example 3:

```
import re

date2 = "45-19-2039"

res1 = re.fullmatch(r"(0[1-9]|1[0-9]|2[0-9]|3[0-1])[-/](0[1-9]|1[0-2])[-/]\d{4}"
,date1)

res2 = re.fullmatch(r"(0[1-9]|[1-2]\d|3[0-1])[-/](0[1-9]|1[0-2])[-/]\d{4}",date1)

if res1 != None:

    print("Valid Date")

else:

    print("Invalid Date")
```

Output: Invalid Date

Explanation:**1. (0[1-9]|1[0-9]|2[0-9]|3[0-1])**

- This part checks for the day (01–31).
- It uses OR | operator to allow different ranges.

a) 0[1-9]

- First digit must be 0
- Second digit can be 1–9

Matches: 01 02 03 04 05 06 07 08 09

b) 1[0-9]

- First digit must be 1
- Second digit can be 0–9

Matches: 10 11 12 13 14 15 16 17 18 19

c) 2[0-9]

- First digit must be 2
- Second digit can be 0–9

Matches: 20 21 22 23 24 25 26 27 28 29

d) 3[0-1]

- First digit must be 3
- Second digit can be 0 or 1

Matches: 30 31

So the whole part matches days from 01 to 31.

2. [-/]:

- This checks the separator between day and month. Match - or /
- Examples: 12-05-2024 , 12/05/2024

3. (0[1-9]|1[0-2]):

- This part checks the month (01–12).

a) 0[1-9]: Matches months: 01 02 03 04 05 06 07 08 09

b) 1[0-2]: Matches months 10 11 12

4. [-/]:

- Again checks the separator between month and year.
- Example: 12-05-2024 , 12/05/2024

5. \d{4}:

- This checks the year.
 - \d → any digit (0–9)
 - {4} → exactly 4 digits

Examples: 1999, 2024 ,2035
